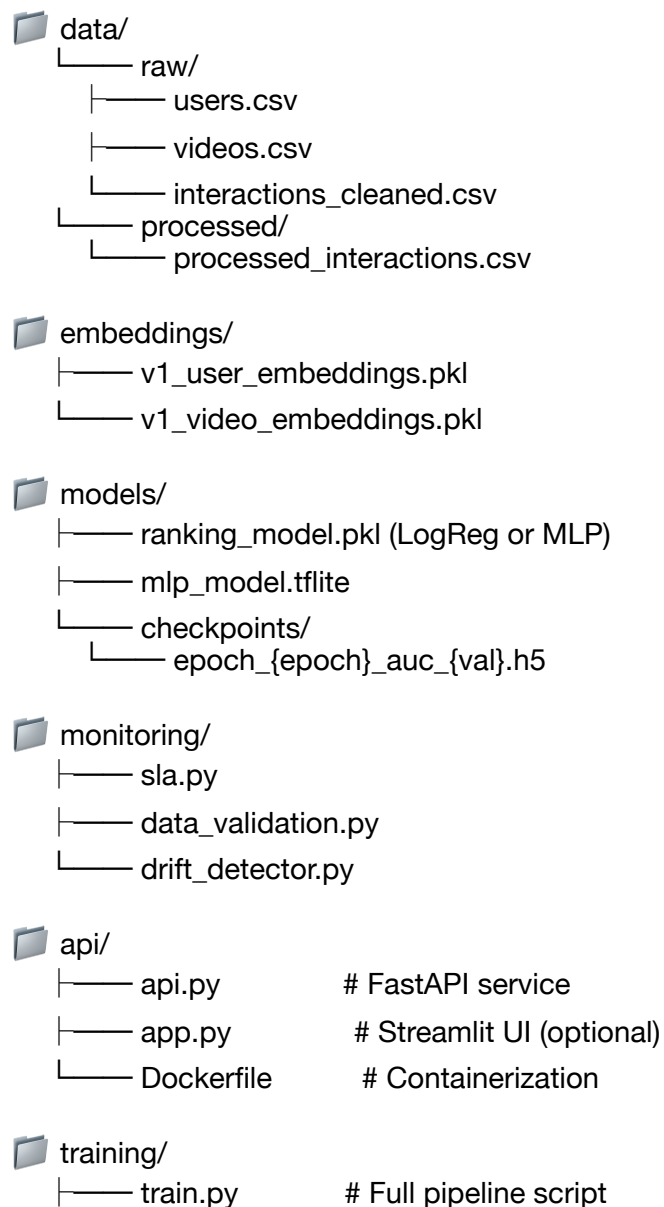


YouTube Shorts Recommendation System — Code Architecture Overview

High-Level System Flow:

1. Data Preprocessing & Feature Engineering
2. Candidate Generation
3. Ranking Model
4. Evaluation
5. Deployment (API + UI)
6. Monitoring & CI/CD

Detailed Architecture Breakdown:



├── model_card.html
└── requirements.txt

 .github/workflows/
├── train.yml # Weekly retraining
└── mlops.yml # Drift-triggered retraining

Core Components:

Candidate Generation (Section 15)

- FAISS GPU Index for nearest neighbor retrieval
- Cold-start fallback: brute-force cosine similarity
- Brings top-N candidates (recall optimized)

Ranking Model (Section 16)

- TensorFlow MLP
- Embedding concatenation: [user] + [video]
- Mixed precision, distributed training
- Evaluated via AUC, NDCG@10

Feature Engineering (Sections 5–7)

- Temporal, recency, session, and popularity features
- One-hot encoding, normalization

Evaluation (Section 12)

- Time-decayed AUC
- NDCG@10 (business-weighted)
- Recall@K
- Significance testing (p-value)

Deployment

- FastAPI + Redis + RateLimiter (Section 24)
- Streamlit UI for demo (Section 25)
- Prometheus + JSON logging (Section 26)
- Docker + Healthcheck endpoint

Monitoring

- SHAP Explainability (Section 18)
- Drift Detection (KSDrift, Section 27)
- SLA Validation (Section 21)

MLOps

- MLflow Tracking (Section 13)
- Versioned models
- Auto-retraining via GitHub Actions + drift check (Section 29)
- Feature store with Feast (Section 30)